RAPPORT D'AUDIT DE PERFORMANCE

American Blackjack

Moteur de simulation de table à haut débit en Go

PÉRIMÈTRE FONCTIONNEL

Moteur de simulation d'une table de blackjack américain : carte cachée avec contrôle du croupier, sabot de 4 jeux et carte de coupe, doubler, séparer jusqu'à quatre mains, abandon tardif, assurance, et les quatre paris annexes du casino. Le moteur ne sert pas à jouer un coup, mais à en dérouler des dizaines de millions pour mesurer l'avantage de la maison sur un jeu de règles donné. La métrique de pilotage est le **débit**, en coups par seconde.

La performance est ici un besoin, pas un confort. L'écart-type du résultat par coup vaut 1,14 unité de mise, soit plus de trois fois l'avantage à mesurer ($\approx 0{,}35$ %). L'erreur-type décroissant en $1/\sqrt{n}$, le résoudre à 0,01 point près exige $\approx 1{,}3 \times 10^8$ coups — soit 48 minutes par exécution sur la version initiale. **Le débit conditionne la précision statistique atteignable.**

Le document couvre la métrologie et les résultats chiffrés, puis le diagnostic matériel et les choix d'ingénierie retenus, y compris une optimisation rejetée après mesure.

DÉPÔT & CADRE

`https://github.com/MaximeHer/american-blackjack`

Tag de référence `v0-baseline` • branche d'échec `echec/memoisation-total`

Maxime H. • Master 2 Expert en Développement Logiciel — M2 Dev B • Octobre 2026

1.1 Banc d'essai et conditions de mesure

Composant	Spécification	Implication
Processeur	AMD Ryzen 5 5600H — 6 cœurs / 12 threads @ 3,30 GHz (Zen 3)	Processeur *mobile* : fréquence dépendante de l'alimentation et de la charge
Caches	L1 384 Ko (64 Ko/cœur) • L2 3072 Ko (512 Ko/cœur) • L3 16 Mo partagé • ligne 64 o	Un sabot compacté (208 o) tient intégralement en L1
Mémoire	16 Go DDR4-3200, **une seule barrette**	**Simple canal** : bande passante divisée par deux, plafonne le passage à l'échelle
Système	Windows 11 build 26200 • Go 1.26.4 `windows/amd64` • GOMAXPROCS = 12	Compilateur `gc`, escape analysis et inlining actifs
Protocole	`hyperfine --shell=none --warmup 3 --runs 15` • `benchstat -benchtime 2s -count 8`	Médiane, écart-type et p-value sur chaque comparaison

Conditions d'exécution. Toutes les mesures retenues ont été prises sur une machine **branchée sur secteur**, **redémarrée juste avant la campagne** et **sans aucune application tierce lancée** — navigateur, éditeur et messagerie fermés, charge CPU vérifiée sous 5 % avant chaque relevé. Le harnais refuse d'ailleurs de démarrer sur batterie et avertit au-delà de ce seuil.

Deux biais ont faussé les premiers relevés (détaillés en 3.3) : une campagne sur machine chargée, et des benchmarks sans puits d'accumulation, dont le compilateur supprimait une partie du calcul mesuré. La baseline a donc été **remesurée depuis son tag Git**, pour que les deux bornes de comparaison partagent la même méthodologie.

1.2 Paliers d'optimisation et effets mesurés

Neuf paliers, chacun isolé dans son commit, mesuré par `benchstat` contre l'état précédent et validé par l'oracle avant d'être conservé.

#	Palier	Cible attaquée	Effet
V1	Journal narratif rendu explicite	≈ 10 `fmt.Sprintf` par coup, produits même en simulation	−47 %
V2	Paris annexes en énumération entière	`map[string]float64` sur le chemin chaud	≈ 0 %
V3	Stratégie dévirtualisée	Appel dynamique par interface	−11 %
V4	Mélange de Fisher-Yates en place	$O(n^2)$: 10 756 déplacements par rebattage	−42 %
V5	**Carte compactée sur 1 octet**	`Card` de 32 o par pointeur + `map[string]`	−57 %
V6	Réordonnancement des champs	19 o et 28 o de remplissage	−6 %
V7	**Table de stratégie plate**	`Sprintf` + hachage : 35 % du temps total	−50 %
V8	**Mains en tableaux fixes (pile)**	4 dernières allocations par coup	−36 %
V9	**Worker pool borné**	1 cœur utilisé sur 12	×4,75

V2 est conservé bien qu'improductif, et illustre la loi du maillon faible : le gain local était réel (−11,6 % sur l'évaluation des paris annexes) mais ceux-ci ne pesaient que 4,4 % du coût d'un coup. Gagner 11,6 % de 4,4 % donne 0,5 %, sous le bruit de mesure.

1.3 Représentation graphique

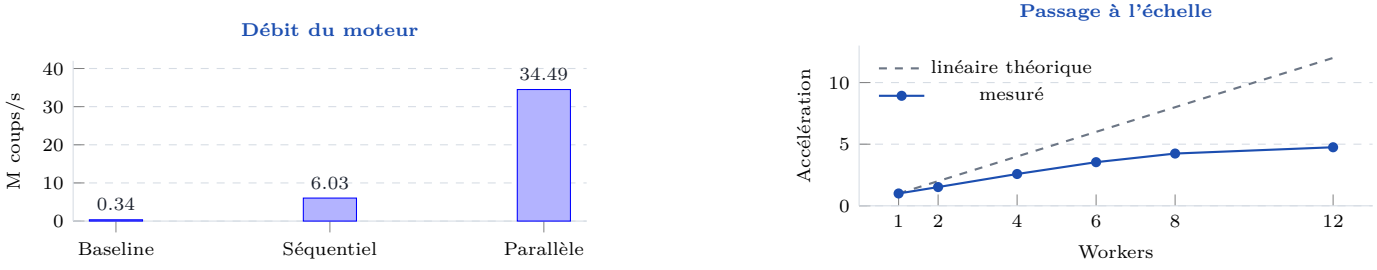


Figure 1 — Débit comparé, et accélération multi-cœurs mesurée contre accélération linéaire théorique.

Le passage à l'échelle est sous-linéaire, pour deux raisons matérielles. Jusqu'à six workers, les cœurs physiques se partagent la bande passante mémoire — une seule barrette, donc un seul canal. Au-delà, ils se partagent les unités de calcul d'un même cœur physique : l'hyperthreading masque des latences mais ne double pas le silicium, et n'apporte

que ×1,34 supplémentaire.

1.4 Validation statistique par benchstat

Comparaison du tag `v0-baseline` à la version finale, **même protocole et même méthodologie de puits des deux côtés**. Toutes les lignes ressortent à $p = 0{,}000$ ($n = 8$).

Benchmark	Baseline	Finale	Gain	Allocs V0	Allocs V9
`PlayRound`	3 731 ns	**164,5 ns**	−95,6 %	46	**0**
`PlayRoundSideBets`	4 563 ns	238,8 ns	−94,8 %	57	**0**
`Decide`	217,8 ns	**9,13 ns**	−95,8 %	3	**0**
`Shuffle`	21 227 ns	1 748 ns	−91,8 %	220	1
`Deal`	97,6 ns	11,0 ns	−88,7 %	1	**0**
`HandTotal`	31,3 ns	3,23 ns	−89,7 %	0	0
`SideBetEval`	87,3 ns	25,6 ns	−70,7 %	0	0
`Simulate (10^4)`	34,05 ms	1,754 ms	−94,9 %	469 800	**344**
`geomean`	**2 274 ns**	**177,9 ns**	−92,2 %		

Reproductibilité en une commande — `bash scripts/run_benchmarks.sh` produit un dossier horodaté : relevé matériel, oracle, benchmarks comparés au tag, mesure `hyperfine`, profils annotés et résumé citant la révision Git mesurée.

Garde-fou de correction. L'avantage de la maison est une grandeur publiée (≈ 0,35 %) que le moteur doit reproduire. L'oracle borne l'écart à quatre erreurs-types calculées depuis la variance observée, et non à une marge arbitraire : il est resté dans la borne sur les neuf paliers, mesuré à +0,4144 % soit **0,80 erreur-type**. Une validation croisée confronte deux implémentations distinctes du règlement, qui concordent à **0,0048 point**.

PARTIE II Diagnostic matériel & choix d'ingénierie

2.1 Identification du chemin chaud

Profilage CPU et allocations par `go tool pprof` : 7,52 s d'échantillons dont 5,48 s dans la boucle de simulation. Trois mécanismes limitants ressortent, et **aucun n'est propre au métier du blackjack** — tous découlent de choix de représentation.

Mécanisme	Part CPU
Hachage de chaînes (`mapaccess`)	27 %
`mallocgc` + `bgsweep`	≈ 40 %
`memmove` / `memclr`	—

Ligne la plus coûteuse	Coût	%
`cardValues[c.Rank]`	1,57 s	20,9
`strategyTable[Sprintf(..)]`	920 ms	12,2
`append(pool, &Card{..})`	640 ms	8,5
`append(pool[:i], pool[i+1:])`	540 ms	7,2

Quatre lignes de code concentrent 49 % du temps d'exécution.

Un reprofilage a invalidé la moitié du plan initial. Les trois premiers paliers avaient été ordonnés *avant* tout profilage. Le profil a montré que le rebattage du sabot, placé en septième position, représentait en réalité **32 % du CPU et 73 % des octets alloués**, tandis que V2 visait une fonction absente du top 16. Un second profil, pris après la compaction des cartes, a de nouveau tout réordonné : le rebattage était tombé à **0,6 %** des allocations et deux paliers planifiés étaient devenus sans objet. **Chaque optimisation ne réduit pas seulement un coût : elle redistribue la rentabilité de toutes les suivantes.**

2.2 Choix d'ingénierie

Axe 1 — Mémoire et localité de cache. Une carte est encodée sur **un seul octet** (4 bits de rang, 2 bits d'enseigne) et stockée par valeur, contre deux chaînes manipulées par pointeur auparavant. Un sabot de 4 jeux passe de **6 656 octets sur 104 lignes de cache plus 208 objets dispersés** à **208 octets contigus sur 4 lignes**, résidents en L1 ; le parcours devient séquentiel, motif que le préchargeur matériel anticipe. Les structures ont été réordonnées par taille décroissante (`Hand` 56 → 40 o, `RoundResult` 80 → 56 o) et les mains placées en **tableaux fixes sur la pile** — une main n'excédant pas 21 cartes et les séparations étant bornées à quatre, tout est connu à la compilation. Résultat : **0 allocation par coup**, vérifié par `go build -gcflags='-m'`.

Axe 2 — Concurrence et scalabilité CPU. Worker pool borné à `runtime.NumCPU()`, jamais une goroutine par tâche : un coup dure 178 ns, là où la création et l'ordonnancement d'une goroutine se comptent en centaines de nanosecondes. La charge étant strictement CPU-bound, le dimensionnement correct est le nombre de cœurs, pas un multiple. Chaque worker accumule dans une **variable locale, sur sa propre pile**, et ne touche à la mémoire partagée qu'une fois, à la fin : deux compteurs voisins dans un tableau partagé tomberaient sur la même ligne de cache de 64 octets, et chaque écriture de l'un invaliderait la ligne dans le cache de l'autre — le **faux partage**, qui annule le gain de parallélisme sans

qu'aucun verrou ne soit en cause.

Le déterminisme survit au parallélisme, condition de validité du protocole : chaque worker reçoit une graine *dérivée* par mélange de bits (un simple `seed + w` corrélerait les séquences voisines) et la fusion se fait dans l'ordre des indices, jamais d'arrivée, l'addition flottante n'étant pas associative.

Axe 3 — I/O réseau et persistance : non traité. L'interface diffuse ses métriques en Server-Sent Events JSON, sans persistance. Flux binaire compact et indexation en base sont la suite identifiée : signaler la lacune valait mieux que la combler superficiellement.

2.3 Confrontation critique : une optimisation rejetée

Hypothèse. Le binaire instrumenté mesure **10,38 appels à `Hand.Total()` par coup**, dont beaucoup sur la même main inchangée — stratégie, test de dépassement et comparaison finale l'interrogent tour à tour. Mettre le résultat en cache devait supprimer l'essentiel de ce travail.

Mesure	Sans cache	Avec cache	Écart
`HandTotal`	3,69 ns	27,65 ns	×7,5
`PlayRound`	255,9 ns	926,3 ns	+261,9 %
Débit	4 210 560 /s	1 151 994 /s	−72,6 %
Objets vivants sur le tas	22 097	387 523	×17,5
Part CPU du ramasse-miettes	4,29 %	11,31 %	×2,6

Cause physique, en deux temps. Consulter la map exige de construire une clé, de la hacher, de sonder le seau puis de comparer la clé complète : 27,65 ns contre 3,69 ns pour le calcul évité — **le cache coûte 7,5 fois plus cher que ce qu'il remplace**. Ensuite, la clé étant la suite ordonnée des cartes, l'espace des clés est immense et rien n'est purgé : le ramasse-miettes trace 387 000 objets vivants de plus par cycle.

L'enseignement est contre-intuitif : c'est *notre propre optimisation précédente* qui a rendu celle-ci perdante. Sur la version de référence, où `Total` coûtait 31,27 ns, un cache à 27,65 ns aurait été un gain ; la compaction des cartes l'a fait tomber à 3,69 ns, et la même idée est passée de bénéfique à catastrophique. L'échec était de plus **indétectable autrement que par la mesure** : la correction restait préservée et tous les tests passaient. Branche `echec/memoisation-total` conservée, non fusionnée.

BILAN Synthèse comparative et arbitrages

3.1 Baseline contre version finale

Indicateur	Baseline (V0)	Finale (V9)	Gain
Débit séquentiel	342 306 coups/s	6 026 483 coups/s	×17,6
Débit parallèle (12 workers)	—	**34 485 553 coups/s**	×100,7
Temps par coup	2 921 ns	**31 ns**	−98,9 %
Coût d'un coup (`PlayRound`)	3 731 ns	164,5 ns	−95,6 %
Allocations par coup	46	**0,03**	−99,9 %
Octets par coup	1 657 o	150 o	−91,0 %
Cycles de GC (2 M coups)	244	4	−98,4 %
Part CPU du ramasse-miettes	15,25 %	**0,00 %**	**éliminée**
Parallélisme effectif (séquentiel)	1,32 cœur	1,01 cœur	**GC hors des cœurs**
Taille d'une carte	32 o	**1 o**	×32
Durée de la suite de tests	17,5 s	1,6 s	−90,9 %

3.2 Sur quoi l'effort a porté

L'accent a porté sur la mémoire avant le CPU, délibérément : le profil initial montrait un moteur limité non par la logique du blackjack mais par trois mécanismes du runtime — hachage de chaînes, allocation sur le tas, déplacements de mémoire. Aucune des quatre lignes les plus coûteuses n'exprimait une règle du jeu.

La parallélisation n'est intervenue qu'en dernier, volontairement : paralléliser un code qui alloue multiplie la pression sur le ramasse-miettes par le nombre de cœurs. Le parallélisme effectif à 1,01 cœur en séquentiel prouve que celui-ci ne mobilisait plus aucun autre cœur avant l'arrivée des workers.

Le palier le plus rentable est la compaction de la carte sur un octet (V5). Il a amélioré *tous* les benchmarks simultanément — de −57 % sur `PlayRound` à −89 % sur `HandTotal` — parce qu'il attaquait deux causes d'un coup : la consultation de map pour la valeur d'une carte (20,9 % du temps total) et l'allocation des 208 cartes à chaque rebattage. Il a de plus débloqué tous les paliers suivants : on ne peut pas indexer un tableau par le rang d'une carte sans que ce rang soit numérique.

Le meilleur rapport effort/gain revient toutefois à V1, le retrait du journal narratif du chemin de simulation :

une quarantaine de lignes pour −47 % et 31 des 46 allocations supprimées. Ce journal n'était pas inutile — l'interface l'affiche — mais il était produit *sans que l'appelant l'ait demandé* : la forme la plus répandue de gaspillage en production, un code partagé entre deux usages payant le coût du plus exigeant.

3.3 Ce que l'audit a coûté en fausses pistes

Écart	Conséquence	Ampleur
Ordre établi sans profil	Deux paliers sur trois dépensés sur des cibles secondaires ; le poste à 32 % du CPU placé en septième position	**2 paliers**
Mesure unique sur machine chargée	Baseline fausse : le bruit dépassait le signal de plusieurs paliers	×1,9
Benchmarks sans puits d'accumulation	Calculs partiellement éliminés par le compilateur, gains surestimés	+20 %
Mémoïsation de `Hand.Total`	Optimisation rejetée après mesure, branche conservée	−72,6 %

Conclusion opérationnelle. Le moteur simule aujourd'hui **34,5 millions de coups par seconde** contre 342 306 au départ. Concrètement, mesurer l'avantage de la maison à 0,01 point près demandait **48 minutes** sur la version initiale ; il faut désormais **moins de 4 secondes**. L'optimisation n'a pas rendu le programme « plus rapide » dans l'abstrait : elle a rendu une mesure statistique **praticable**. L'avantage de la maison mesuré n'a jamais dévié de plus de 0,80 erreur-type de la valeur publiée sur l'ensemble des neuf paliers.

ANNEXE Gouvernance technique IA — `constitution.md`

Un fichier `constitution.md` à la racine du dépôt prime sur toute suggestion par défaut d'un assistant de génération de code.

1 — Rôle et posture système. L'assistant y est défini comme ingénieur systèmes de performance, raisonnant en cycles CPU, lignes de cache, défauts de cache et pression du ramasse-miettes, jamais en lisibilité. *« Un gain sans mesure est un échec. Une mesure sans protocole est du bruit. »*

2 — Contraintes négatives explicites. Sont proscrits sur le chemin chaud : `fmt.Sprintf` et toute mise en forme de chaîne ; toute allocation sur le tas non justifiée par `-benchmem` ; la `map` comme table de correspondance à domaine borné ; les goroutines non bornées ; le `math/rand` global ; `interface{}` et la réflexion ; tout verrou pris dans une boucle serrée.

3 — Justification empirique obligatoire. Toute proposition se formule en trois lignes et aucune autre forme n'est acceptée : `HYPOTHÈSE` (effet physique attendu, quantifié), `VÉRIFICATION` (commande exacte), `RÉFUTATION` (résultat qui invaliderait l'hypothèse). Les comparaisons passent par `benchstat`, jamais par deux nombres bruts ; toute mesure dont la variance dépasse 5 % est rejetée.

4 — Formatage impératif, et obligation de documenter l'échec — une optimisation qui régresse n'est pas supprimée mais consignée, chiffrée, expliquée mécaniquement, puis annulée.

Code source, mesures brutes, profils et documentation détaillée : `https://github.com/MaximeHer/american-blackjack` (dossier `docs/`).